UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II
Corso di Laurea Magistrale in Ingegneria Informatica
Corso di Network and Cloud Infrastructures — Prof. Giorgio Ventre
Anno Accademico 2025/2026

SDN Project Work: Network Slicing with Ryu and Mininet

Studenti partecipanti al progetto

Starita Gianluigi Matr. DE90049

Saturno Jacopo Matr. DE9000141

Introduzione

Il paradigma **SDN (Software-Defined Networking)** separa il piano di controllo dal piano dati, centralizzando l'intelligenza di rete in un controller software che programma gli switch tramite il protocollo **OpenFlow**. Ciò abilita il **Network Slicing**, ovvero la suddivisione di un'unica rete fisica in più reti virtuali (*slice*) isolate e ottimizzate per specifici utenti o servizi. Il progetto emula un'architettura SDN con **Mininet** e il controller **Ryu** (OpenFlow 1.3), collegando quattro host (H1–H4) tramite due switch di accesso (S1, S4) e due percorsi di backbone a diversa capacità: una **Upper Slice** a `10 Mbps` (S2) e una **Lower Slice** a `1 Mbps` (S3).

Obiettivi del Progetto

L'obiettivo è implementare e confrontare due diverse politiche di Network Slicing sulla stessa topologia fisica:

- **Infrastruttura di rete (`topology.py`):** Creare su Mininet la topologia con vincoli di banda e ritardo, collegando gli switch al controller remoto Ryu.
- **Slicing Statico (`static_slicing.py`):** Isolare rigidamente la rete a Livello 2 (MAC/DPID) in due slice dedicate: H1-H3 su S2 (`10 Mbps`) e H2-H4 su S3 (`1 Mbps`) bloccando con regole di *DROP* ogni comunicazione incrociata o percorso non autorizzato.
- **Slicing per Servizio (`service_slicing.py`):** Consentire la comunicazione tra tutti gli host, ma smistare dinamicamente il traffico fino al Livello 4, instradando i flussi video (UDP porta 9999) sulla *Upper Slice* veloce (S2, `10 Mbps`) e il traffico standard sulla *Lower Slice* (S3, `1 Mbps`).

Topology Slicing

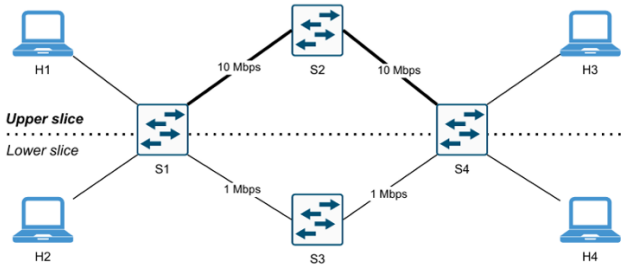
Il sistema implementa una separazione logica tra due differenti *slice* di rete. La prima comprende gli host H1 e H3 (slice superiore), mentre la seconda include gli host H2 e H4 (slice inferiore). Le

comunicazioni sono consentite esclusivamente tra gli host appartenenti alla stessa slice, mentre tutti gli altri flussi di traffico vengono bloccati.

Il controller di rete garantisce il rispetto di tale isolamento attraverso diversi meccanismi: il rilevamento delle violazioni tra slice, l'apprendimento dinamico degli indirizzi MAC, il filtraggio dei pacchetti ARP in base alla slice di appartenenza e la gestione del traffico mediante le funzionalità *FlowMod* e *PacketOut* previste dal protocollo OpenFlow.

Utilizza l'emulatore **Mininet** per costruire l'infrastruttura fisica virtualizzata (Piano Dati) mostrata in `topology.png`, collegandola al controller esterno Ryu:

- **Inizializzazione dell'Ambiente:** Crea una rete Mininet configurata con switch *Open vSwitch* in modalità kernel (`OVSKernelSwitch`), collegamenti a prestazioni controllate (`TCLink`) e un controller esterno (`RemoteController`, che si connette automaticamente a Ryu).
- **Creazione dei Nodi:** Istanzia **4 host** (h1–h4) assegnando loro indirizzi MAC (`00:00:00:00:00:01–04`) e IP (`10.0.0.1–04`) statici, e **4 switch** (s1–s4, con DPID da 1 a 4) forzati a usare il protocollo **OpenFlow 1.3**.
- **Collegamenti di Accesso (Host $\leftrightarrow$ Switch):** Collega h1 e h2 allo switch di accesso `s1` (porte 1 e 2) e h3 e h4 allo switch di accesso `s4` (porte 1 e 2), tutti con banda di **10 Mbps** e ritardo trascurabile (`0.0025ms`).
- **Collegamenti di Backbone (Le due Slice):** Crea due percorsi paralleli tra `s1` e `s4` con ritardo di `25ms` su ciascun tratto ma capacità differenti:
  - **Upper Slice (Alta banda):** Collega `s1` $\leftrightarrow$ `s2` $\leftrightarrow$ `s4` (usando la porta 3 su `s1` e `s4`) con una banda di **10 Mbps**.
  - **Lower Slice (Bassa banda):** Collega `s1` $\leftrightarrow$ `s3` $\leftrightarrow$ `s4` (usando la porta 4 su `s1` e `s4`) con una banda limitata a **1 Mbps**.
- **Avvio e CLI:** Compila e avvia la rete (`net.build()`, `net.start()`) e apre l'interfaccia a riga di comando interattiva di Mininet (`CLI`) per eseguire i test di traffico tra gli host.



Static Slicing

Questo controller Ryu divide rigidamente la rete in due slice isolate a livello 2 (MAC) associando in modo fisso gli host ai percorsi di backbone:

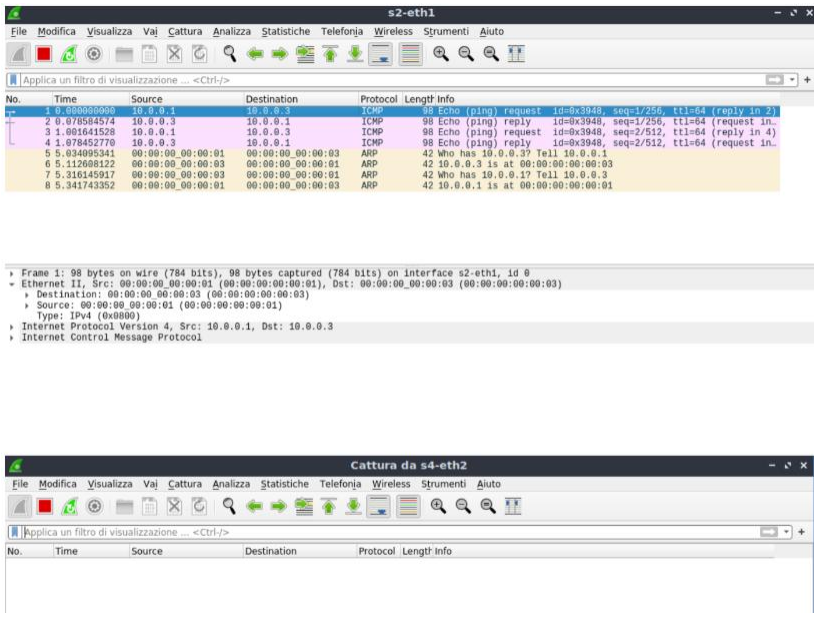
- **Definizione delle Slice:** Crea una **Upper Slice** (10 Mbps) riservata esclusivamente alla coppia **H1 <-> H3** tramite lo switch **S2**, e una **Lower Slice** (1 Mbps) riservata esclusivamente alla coppia **H2 <-> H4** tramite lo switch **S3**.

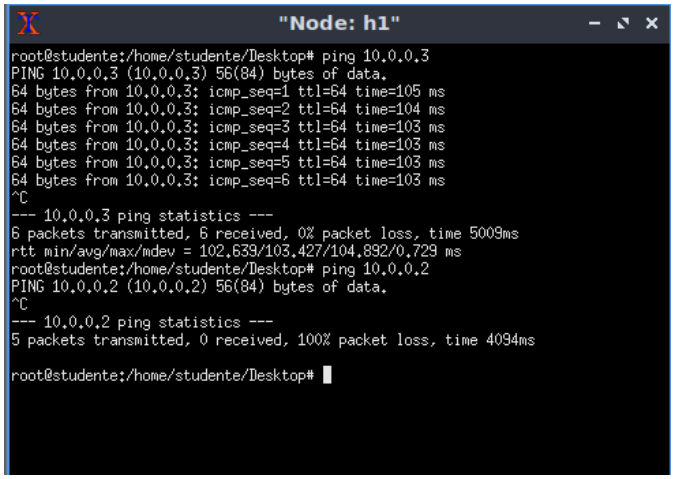
- **Inizializzazione (Priorità 0):** Alla connessione di ogni switch (`switch_features_handler`), installa una regola di default (*Table-Miss*) a **priorità 0** che invia al controller tutti i pacchetti non ancora conosciuti.
- **Apprendimento e Filtro ARP:** Per ogni pacchetto ricevuto (`_packet_in_handler`), ignora il traffico LLDP e memorizza la porta di provenienza del MAC sorgente (*MAC learning*). Se il pacchetto è una richiesta/risposta **ARP**, la propaga in *flooding* solo sugli switch appartenenti alla slice corretta, bloccandola invece con una regola di **DROP a priorità 100** se finisce sullo switch di backbone sbagliato (`violates_slice`).
- **Blocco delle Violazioni (Priorità 100):** Per il traffico dati (es. ping/IPv4), se un pacchetto transita sullo switch di backbone errato (H1/H3 su S3, oppure H2/H4 su S2) o se tenta una comunicazione incrociata non presente nella lista `self.allowed` (es. H1 verso H2 o H4), il controller installa immediatamente sullo switch una regola di **DROP** (`actions = []`) con **priorità 100**.
- **Inoltro Veloce (Priorità 10):** Se il traffico rispetta sia la coppia di host autorizzata sia il percorso di backbone corretto, il controller inoltra il pacchetto sulla porta di uscita associata al MAC di destinazione e installa una regola permanente a **priorità 10**, permettendo ai pacchetti successivi di viaggiare direttamente attraverso gli switch senza ripassare dal controller.

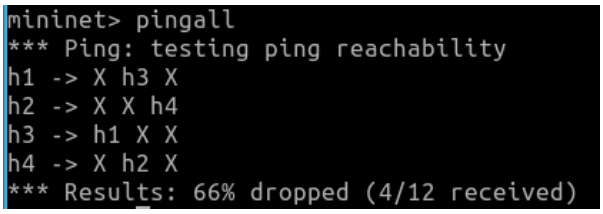
L'ispezione delle flow entry presenti nella tabella di inoltro dello switch S1 conferma che il controller SDN genera e installa regole esclusivamente per le comunicazioni ammesse all'interno della stessa slice, garantendo così l'isolamento logico tra i diversi domini di rete:

```
studente@studente:~$ sudo ovs-ofctl -O OpenFlow13 dump-flows s1
[sudo] password di studente:
 cookie=0x0, duration=112.659s, table=0, n_packets=5, n_bytes=470, priority=100,in_port="s1-eth3",dl_src=ce:f7:d4:d1:a1:c5,dl_dst=33:33:00:00:00:16 actions=drop
 cookie=0x0, duration=112.627s, table=0, n_packets=0, n_bytes=0, priority=100,in_port="s1-eth2",dl_src=c2:c8:db:0b:79:4d,dl_dst=33:33:00:00:00:16 actions=drop
 cookie=0x0, duration=112.610s, table=0, n_packets=0, n_bytes=0, priority=100,in_port="s1-eth1",dl_src=06:62:eb:5f:86:4e,dl_dst=33:33:ff:5f:86:4e actions=drop
 cookie=0x0, duration=112.597s, table=0, n_packets=5, n_bytes=470, priority=100,in_port="s1-eth4",dl_src=8a:97:fc:e9:7a:66,dl_dst=33:33:00:00:00:16 actions=drop
 cookie=0x0, duration=112.539s, table=0, n_packets=3, n_bytes=290, priority=100,in_port="s1-eth1",dl_src=00:00:00:00:00:01,dl_dst=33:33:00:00:00:16 actions=drop
 cookie=0x0, duration=112.526s, table=0, n_packets=3, n_bytes=290, priority=100,in_port="s1-eth2",dl_src=00:00:00:00:00:02,dl_dst=33:33:00:00:00:16 actions=drop
 cookie=0x0, duration=112.402s, table=0, n_packets=0, n_bytes=0, priority=100,in_port="s1-eth2",dl_src=00:00:00:00:00:02,dl_dst=33:33:ff:00:00:02 actions=drop
 cookie=0x0, duration=112.290s, table=0, n_packets=0, n_bytes=0, priority=100,in_port="s1-eth4",dl_src=8a:97:fc:e9:7a:66,dl_dst=33:33:ff:e9:7a:66 actions=drop
 cookie=0x0, duration=111.936s, table=0, n_packets=0, n_bytes=0, priority=100,in_port="s1-eth3",dl_src=ce:f7:d4:d1:a1:c5,dl_dst=33:33:ff:d1:a1:c5 actions=drop
 cookie=0x0, duration=111.542s, table=0, n_packets=0, n_bytes=0, priority=100,in_port="s1-eth1",dl_src=00:00:00:00:00:01,dl_dst=33:33:ff:00:00:01 actions=drop
 cookie=0x0, duration=111.386s, table=0, n_packets=4, n_bytes=280, priority=100,in_port="s1-eth2",dl_src=00:00:00:00:00:02,dl_dst=33:33:00:00:00:02 actions=drop
 cookie=0x0, duration=111.265s, table=0, n_packets=4, n_bytes=280, priority=100,in_port="s1-eth4",dl_src=8a:97:fc:e9:7a:66,dl_dst=33:33:00:00:00:02 actions=drop
 cookie=0x0, duration=111.163s, table=0, n_packets=12, n_bytes=1890, priority=100,in_port="s1-eth4",dl_src=8a:97:fc:e9:7a:66,dl_dst=33:33:00:00:00:fb actions=drop
 cookie=0x0, duration=110.913s, table=0, n_packets=4, n_bytes=280, priority=100,in_port="s1-eth3",dl_src=ce:f7:d4:d1:a1:c5,dl_dst=33:33:00:00:00:02 actions=drop
 cookie=0x0, duration=110.811s, table=0, n_packets=12, n_bytes=1890, priority=100,in_port="s1-eth3",dl_src=ce:f7:d4:d1:a1:c5,dl_dst=33:33:00:00:00:fb actions=drop
 cookie=0x0, duration=110.519s, table=0, n_packets=4, n_bytes=280, priority=100,in_port="s1-eth1",dl_src=00:00:00:00:00:01,dl_dst=33:33:00:00:00:02 actions=drop
 cookie=0x0, duration=102.317s, table=0, n_packets=0, n_bytes=0, priority=100,in_port="s1-eth1",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:02 actions=drop
 cookie=0x0, duration=92.203s, table=0, n_packets=2, n_bytes=140, priority=10,in_port="s1-eth1",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:03 actions=output:"s1-eth3"
 cookie=0x0, duration=92.090s, table=0, n_packets=2, n_bytes=140, priority=10,in_port="s1-eth3",dl_src=00:00:00:00:00:03,dl_dst=00:00:00:00:00:01 actions=output:"s1-eth1"
 cookie=0x0, duration=91.974s, table=0, n_packets=0, n_bytes=0, priority=100,in_port="s1-eth1",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:04 actions=drop
 cookie=0x0, duration=82.082s, table=0, n_packets=3, n_bytes=126, priority=100,in_port="s1-eth2",dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:01 actions=drop
 cookie=0x0, duration=71.958s, table=0, n_packets=0, n_bytes=0, priority=100,in_port="s1-eth2",dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:03 actions=drop
 cookie=0x0, duration=61.957s, table=0, n_packets=2, n_bytes=140, priority=10,in_port="s1-eth2",dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:04 actions=output:"s1-eth4"
 cookie=0x0, duration=61.846s, table=0, n_packets=2, n_bytes=140, priority=10,in_port="s1-eth4",dl_src=00:00:00:00:00:04,dl_dst=00:00:00:00:00:02 actions=output:"s1-eth2"
 cookie=0x0, duration=147.819s, table=0, n_packets=36, n_bytes=2702, priority=0 actions=CONTROLLER:65535
```

Cattura del traffico sulle interfacce **s2-eth1** e **s4-eth2** in seguito all'esecuzione di un ping tra gli host H1 e H3:







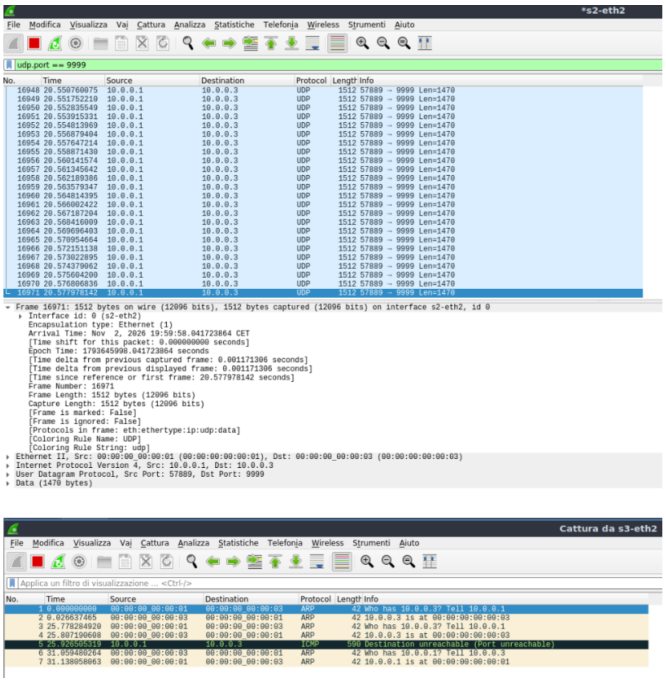
Service Slicing

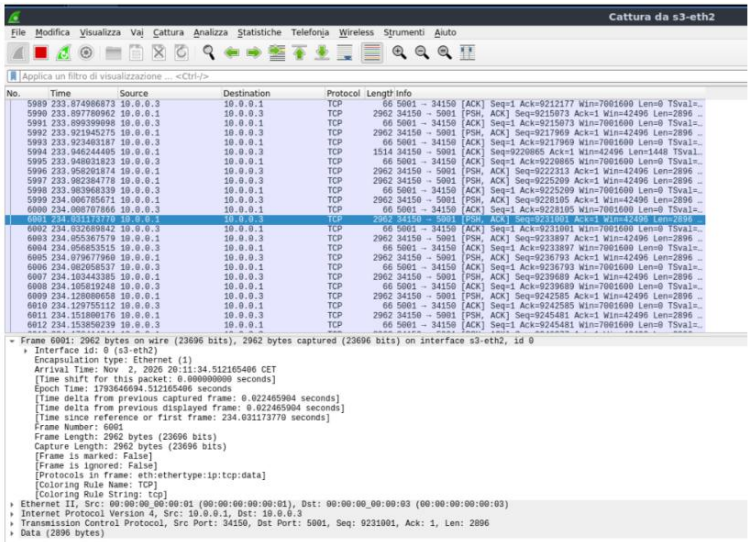
implementa un controller SDN in Ryu (`SliceEnforcingController`) che realizza uno **Slicing Dinamico basato sul Servizio**: consente a tutti gli host di comunicare tra loro, ma separa i flussi di rete sui due rami della backbone in base al tipo di applicazione (Livello 4 UDP).

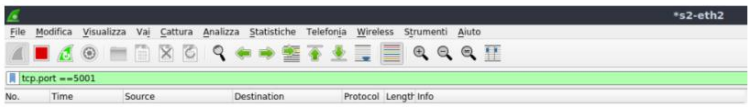
- **Definizione delle Slice per Servizio:** Instrada il **traffico Video** (identificato da pacchetti IPv4 UDP con porta sorgente o destinazione 9999) sulla **Upper Slice** ad alta capacità (S2, collegato alla porta 3 di S1 e S4, da 10 Mbps), mentre convoglia **tutto il resto del traffico** (ARP, ping ICMP, TCP, UDP su altre porte) sulla **Lower Slice** a bassa capacità (S3, collegato alla porta 4 di S1 e S4, da 1 Mbps).
- **Regole Proattive Iniziali (Priorità 0 e 15):** Alla connessione di ogni switch (`switch_features_handler`), oltre alla regola *Table-Miss* di default a **priorità 0**, installa due regole proattive a **priorità 15** che intercettano qualsiasi pacchetto UDP con porta sorgente o destinazione 9999 e lo inviano al controller. Questo garantisce che i pacchetti video scavalchino sempre le regole del traffico normale (a priorità 10) eventualmente già presenti sugli switch.
- **Apprendimento e Classificazione:** Per ogni pacchetto ricevuto (`_packet_in_handler`), registra la porta associata al MAC sorgente (*MAC learning*) ed esamina gli header IPv4 e UDP per impostare il flag booleano `is_video` se la porta UDP è la 9999.
- **Gestione sugli Switch di Accesso (S1 e S4):**
  - **Flooding Selettivo:** Se il MAC di destinazione è sconosciuto, invia il pacchetto alle porte degli host locali (1 e 2) e ad **uno solo** dei rami di backbone: la porta 3 (S2) se è traffico video, oppure la porta 4 (S3) se è traffico normale.
  - **Forzatura della Porta e Regole di Flusso:** Se il MAC di destinazione è già noto ma punta allo switch di backbone sbagliato per quel tipo di servizio (es. un pacchetto video che punta a `PORT_S3` o un pacchetto normale che punta a `PORT_S2`), il controller sovrascrive dinamicamente `out_port` con la porta corretta. Installa poi una regola specifica a **priorità 20** (`PRIORITY_VIDEO`) per il traffico video, oppure una regola L2 a **priorità 10** (`PRIORITY_DEFAULT`) per il traffico standard.

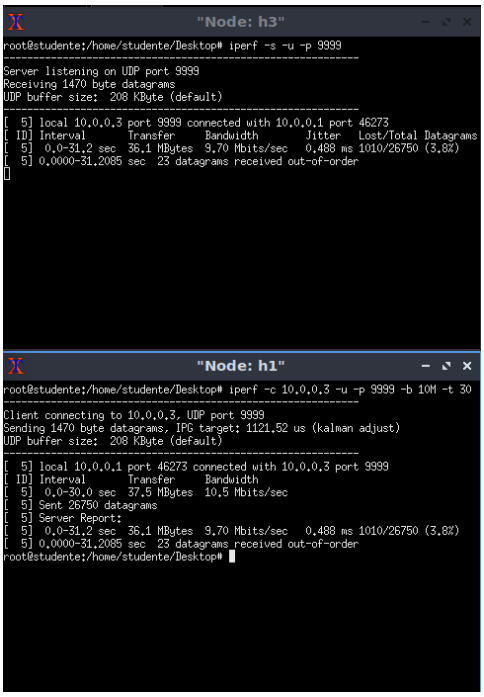
Gestione sugli Switch di Backbone (S2 e S3): Si comportano come normali switch L2 che imparano gli indirizzi MAC e installano regole di inoltro a **priorità 10** verso la porta di destinazione (o eseguono flooding se sconosciuta).

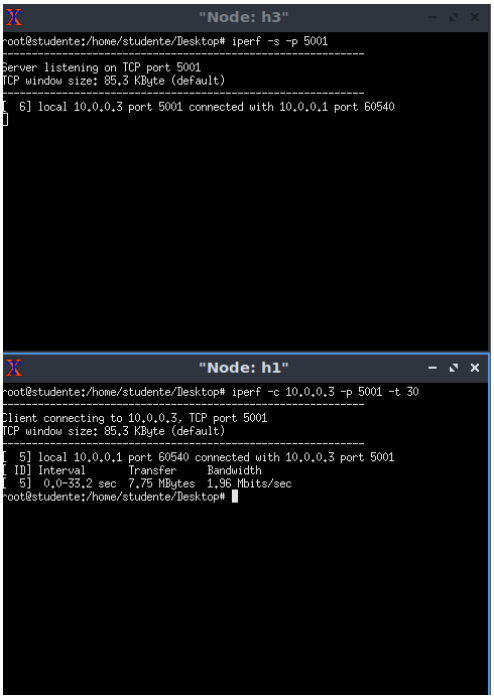
Vediamo di seguito come cambia la gestione del traffico in base alla tipologia di servizio:












```
studente@studente:~$ sudo ovs-ofctl -O OpenFlow13 dump-flows s1
[sudo] password di studente:
 cookie=0x0, duration=343.846s, table=0, n_packets=25745, n_bytes=38926440, priority=20,udp,i
n_port="s1-eth1",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:03,tp_dst=9999 actions=output
:"s1-eth3"
 cookie=0x0, duration=312.507s, table=0, n_packets=4, n_bytes=6048, priority=20,udp,in_port="
s1-eth3",dl_src=00:00:00:00:00:03,dl_dst=00:00:00:00:00:01,tp_src=9999 actions=output:"s1-eth
1"
 cookie=0x0, duration=788.563s, table=0, n_packets=1, n_bytes=1512, priority=15,udp,tp_dst=99
99 actions=CONTROLLER:65535
 cookie=0x0, duration=788.563s, table=0, n_packets=2, n_bytes=3024, priority=15,udp,tp_src=99
99 actions=CONTROLLER:65535
 cookie=0x0, duration=765.211s, table=0, n_packets=3, n_bytes=238, priority=10,in_port="s1-et
h2",dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:01 actions=output:"s1-eth1"
 cookie=0x0, duration=765.209s, table=0, n_packets=2, n_bytes=140, priority=10,in_port="s1-et
h1",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:02 actions=output:"s1-eth2"
 cookie=0x0, duration=765.092s, table=0, n_packets=2436, n_bytes=160632, priority=10,in_port=
"s1-eth4",dl_src=00:00:00:00:00:03,dl_dst=00:00:00:00:00:01 actions=output:"s1-eth1"
 cookie=0x0, duration=765.088s, table=0, n_packets=2897, n_bytes=8317226, priority=10,in_port
="s1-eth1",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:03 actions=output:"s1-eth4"
 cookie=0x0, duration=764.859s, table=0, n_packets=3, n_bytes=238, priority=10,in_port="s1-et
h4",dl_src=00:00:00:00:00:04,dl_dst=00:00:00:00:00:01 actions=output:"s1-eth1"
 cookie=0x0, duration=764.857s, table=0, n_packets=2, n_bytes=140, priority=10,in_port="s1-et
h1",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:04 actions=output:"s1-eth4"
 cookie=0x0, duration=764.626s, table=0, n_packets=3, n_bytes=238, priority=10,in_port="s1-et
h4",dl_src=00:00:00:00:00:03,dl_dst=00:00:00:00:00:02 actions=output:"s1-eth2"
 cookie=0x0, duration=764.624s, table=0, n_packets=2, n_bytes=140, priority=10,in_port="s1-et
h2",dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:03 actions=output:"s1-eth4"
 cookie=0x0, duration=764.393s, table=0, n_packets=3, n_bytes=238, priority=10,in_port="s1-et
h4",dl_src=00:00:00:00:00:04,dl_dst=00:00:00:00:00:02 actions=output:"s1-eth2"
 cookie=0x0, duration=764.390s, table=0, n_packets=2, n_bytes=140, priority=10,in_port="s1-et
h2",dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:04 actions=output:"s1-eth4"
 cookie=0x0, duration=788.563s, table=0, n_packets=262, n_bytes=26472, priority=0 actions=CON
TROLLER:65535
```